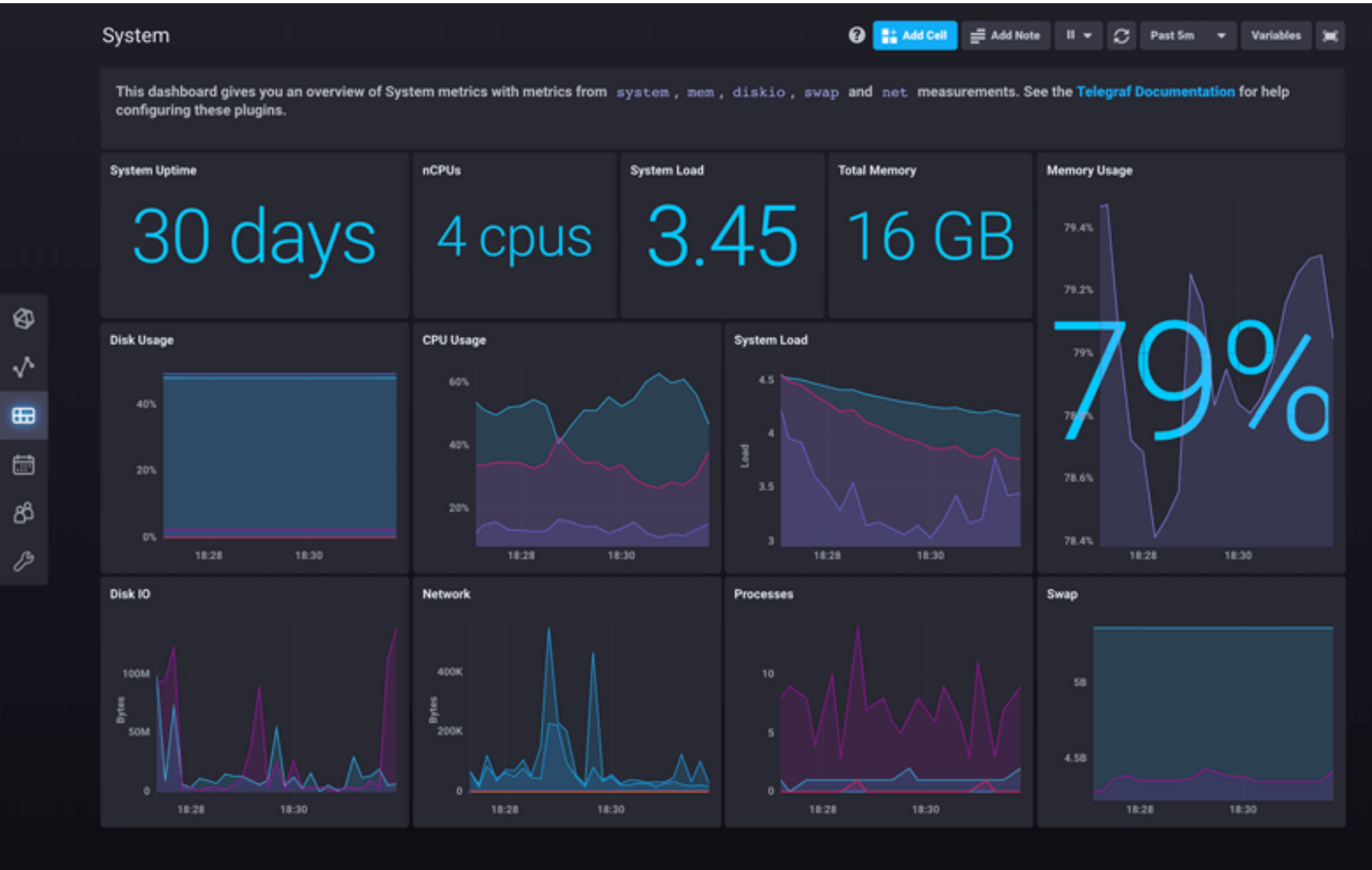


지표 모니터링 및 경보 시스템

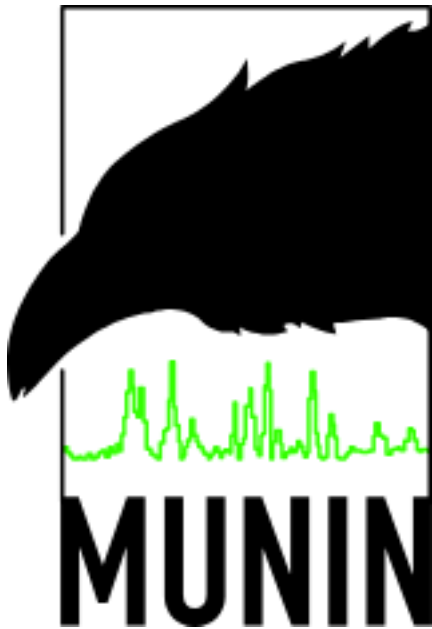
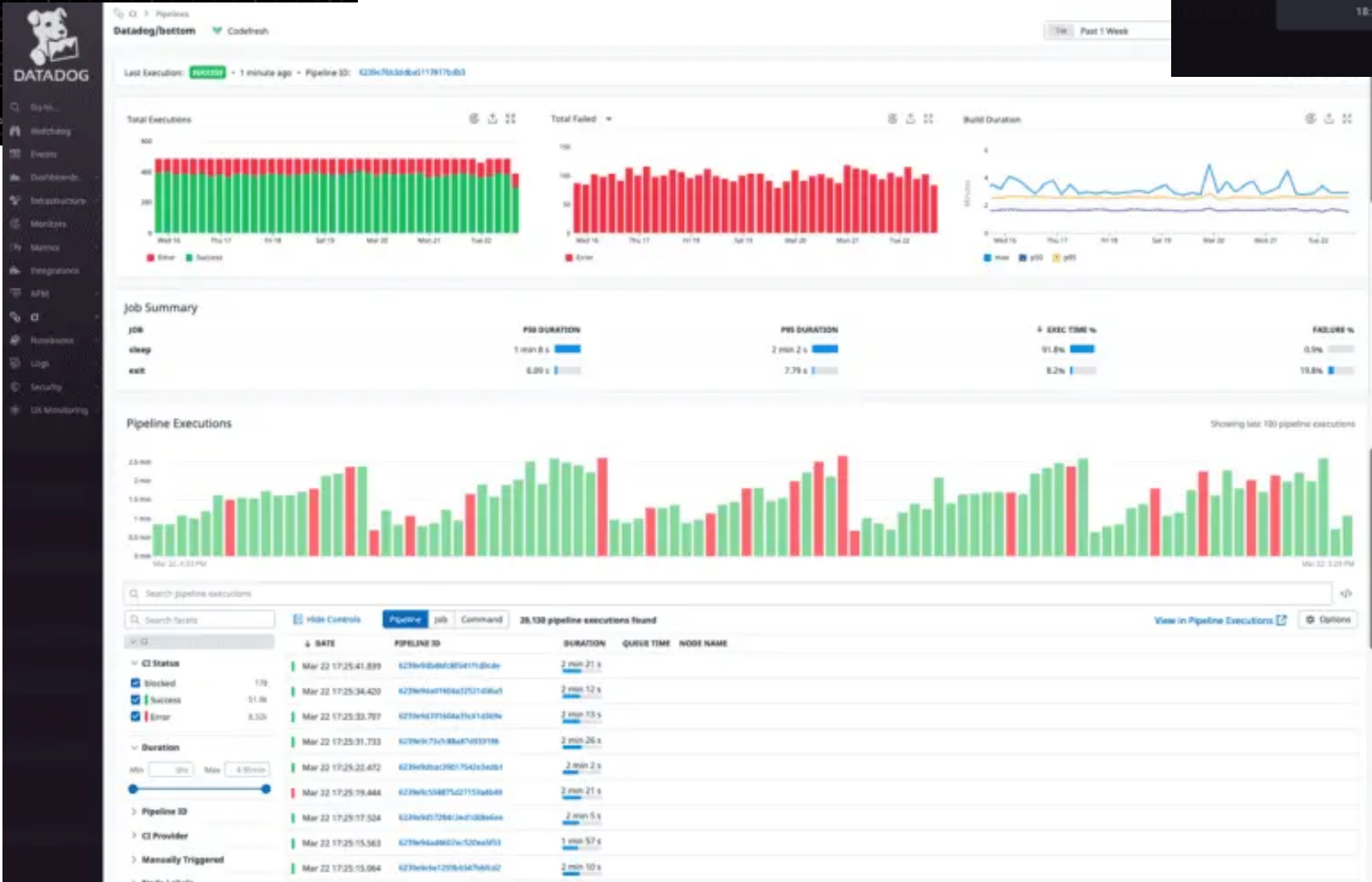
멋사 백엔드 스터디 발표

- 지표 모니터링 및 경보 시스템이란?
- 요구사항
- 개략적 설계안
- 상세 설계
- 마무리

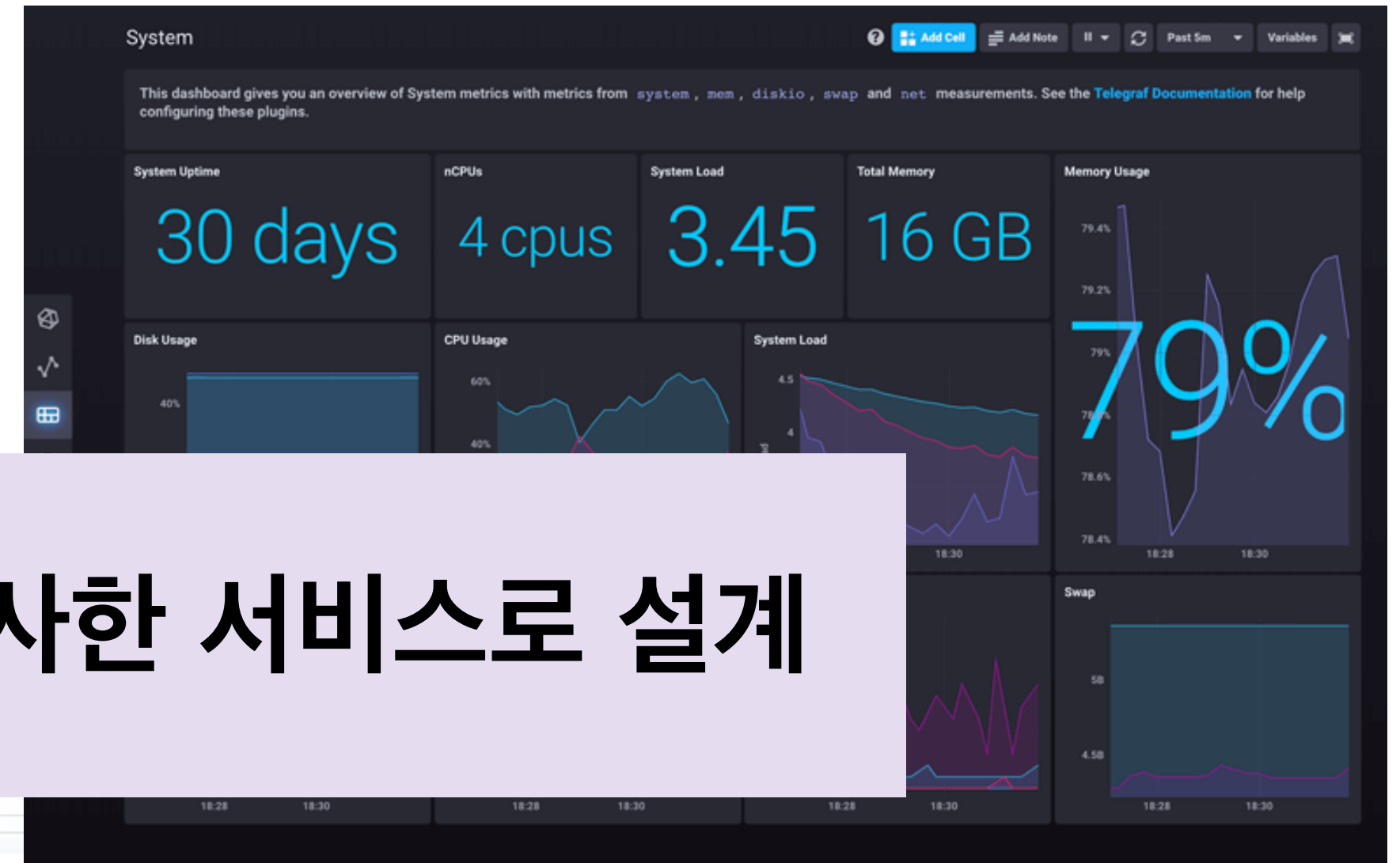
지표 모니터링 및 경보 시스템이 뭔데?



Prometheus



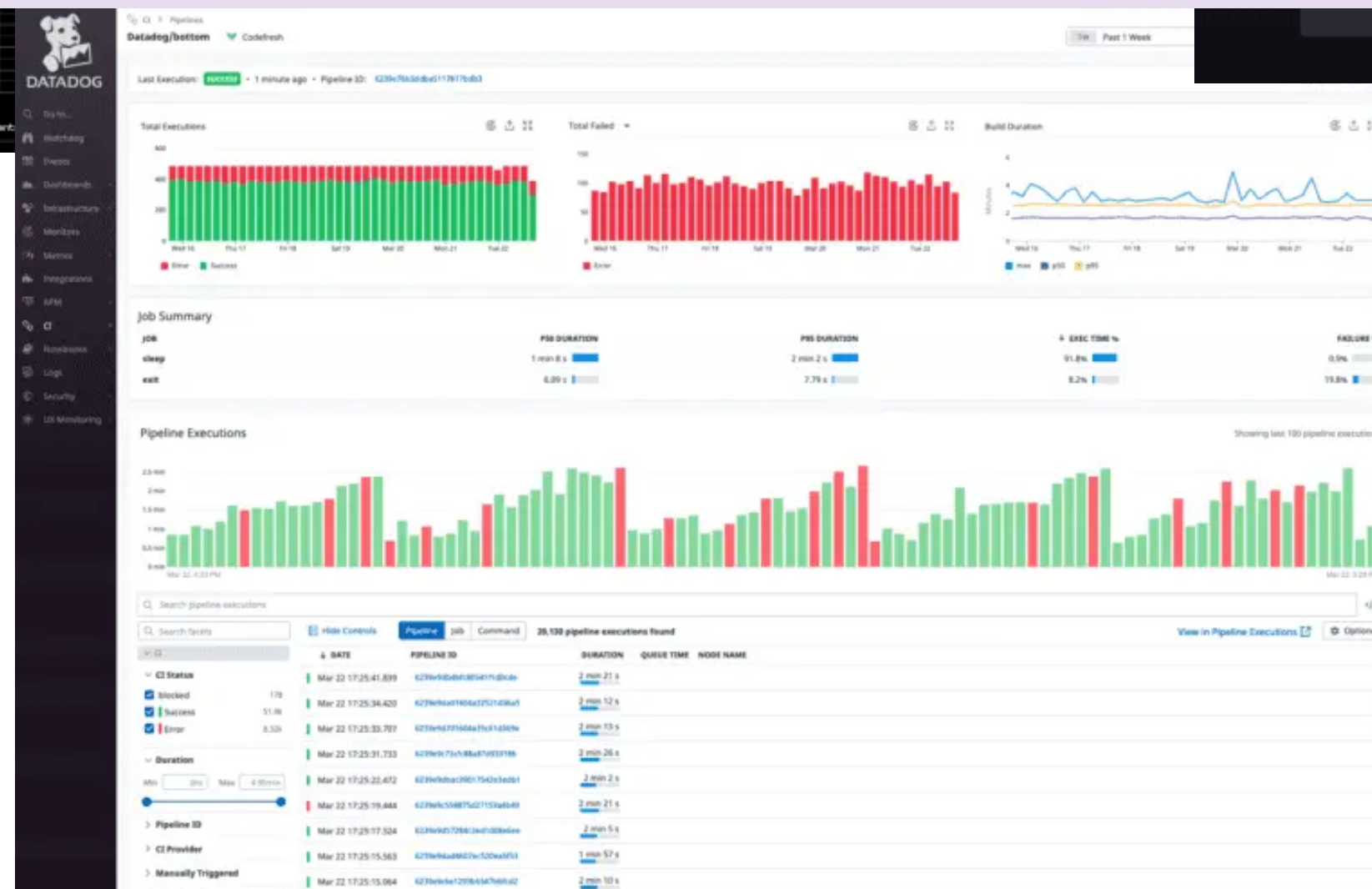
지표 모니터링 및 경보 시스템이 뭔데?



대형 IT 업체에서 사용하는 것과 유사한 서비스로 설계



Prometheus



요구 사항

요구사항

- 대규모 인프라 모니터링
 - 일간 능동 사용자 수 1억 명
 - 서버 풀 1000개, 풀당 서버 수 100개, 서버당 100개의 운영 지표 수집
 - 모니터링 해야 하는 지표의 수 = 천만 개 수준
 - 데이터 보관 기간은 1년
(수집한 그대로 - 1주일, 1분 단위 데이터 변환 - 30일, 1시간 단위 데이터 변환 - 1년 보관)
- 모니터링 할 지표 (예시)
 - CPU 사용률
 - 요청 수
 - 메모리 사용량
 - 메시지 큐 내의 메시지 수
 - 등등

요구사항 (비기능)

- 규모 확장성
- 낮은 응답 지연
- 안정성
- 유연성

고려하지 X → 로그 모니터링, 분산 시스템 추적

개략적 설계안

개략적 설계안

시스템의 다섯 컴포넌트

- 데이터 수집
- 데이터 전송
- 데이터 저장소
- 경고
- 시각화

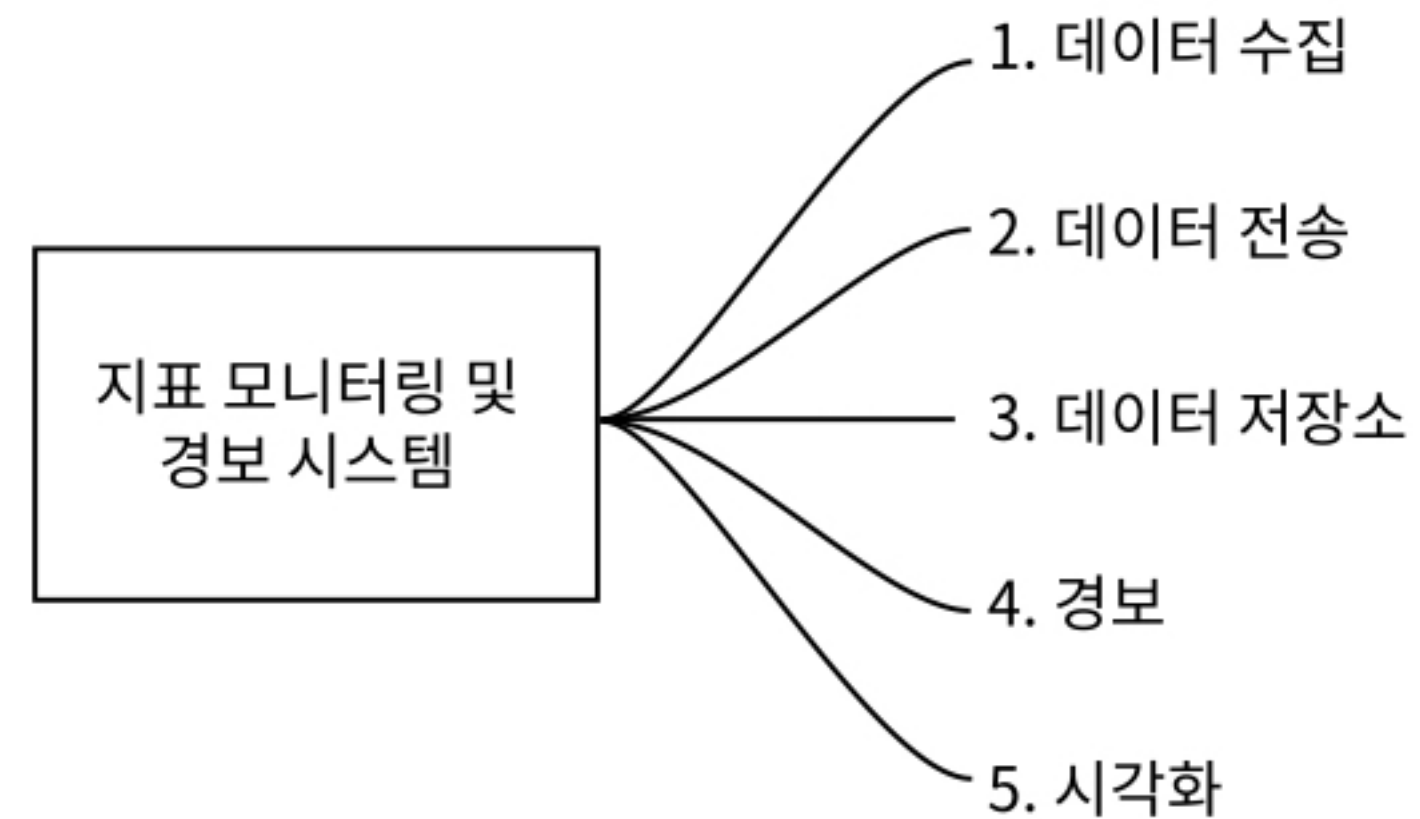


그림 5.2 시스템의 다섯 가지 구성 요소

개략적 설계안

데이터 모델 - 시계열

- 지표 데이터는 시계열 데이터 형태로 기록, 값 집합에 타임스탬프가 붙은 형태로 기록

사례 1

프로덕션에서 사용 중인 서버 인스턴스 i631의 20:00 시점의 CPU 부하를 알고 싶다고 해 보자.

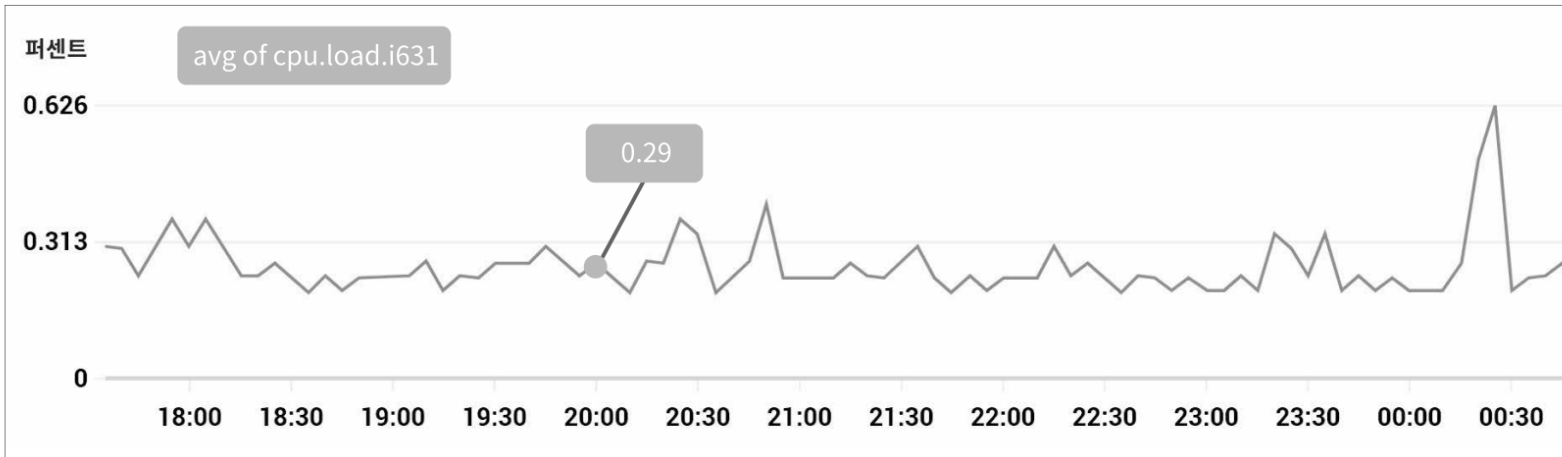


그림 5.3 서버 부하 모니터링 사례

metric_name	cpu.load
labels	host:i631,env:prod
timestamp	1613707265
value	0.29

표 5.1 테이블로 표현한 데이터

이름	자료형
지표 이름	문자열
태그/레이블 집합	<키:값> 쌍의 리스트(List)
지표 값 및 그 타임스탬프의 배열	<값, 타임스탬프> 쌍의 배열(Array)

표 5.2 : 시계열 데이터

사례2

지난 10분간 us-west 지역(region)에 위치한 모든 웹 서버의 CPU 부하 평균값은 얼마인가? 이 질문에 답하려면 개념적으로는 지표 이름이 CPU.load이고 레이블에 포함된 지역 이름이 us-west인 다음과 같은 데이터를 저장소에서 가져와 평균값을 구하면 될 것이다.

```
CPU.load host=webserver01,region=us-west 1613707265 50
CPU.load host=webserver01,region=us-west 1613707265 62
CPU.load host=webserver02,region=us-west 1613707265 43
CPU.load host=webserver02,region=us-west 1613707265 53
...
CPU.load host=webserver01,region=us-west 1613707265 76
CPU.load host=webserver01,region=us-west 1613707265 83
```

행의 데이터 형식 = 행 프로토콜
(프로메테우스, OpenTSDB)

개략적 설계안

데이터 모델 - 데이터 접근 패턴

- 막대한 쓰기 연산 부하
- 일시적으로 치솟았다 사라지는(spiky) 읽기 연산 부하

개략적 설계안

데이터 저장소 시스템 - 시계열 데이터베이스

- RDBMS/NoSQL 사용 비추천
: 높은 쓰기 연산 부하와 집계 질의를 처리하기에 적합하지 않음, 튜닝에 상당한 노력 필요
- 시계열 데이터베이스(TSDB) 사용 권장
- 가장 인기: InfluxDB, 프로메테우스



Prometheus

개략적 설계안

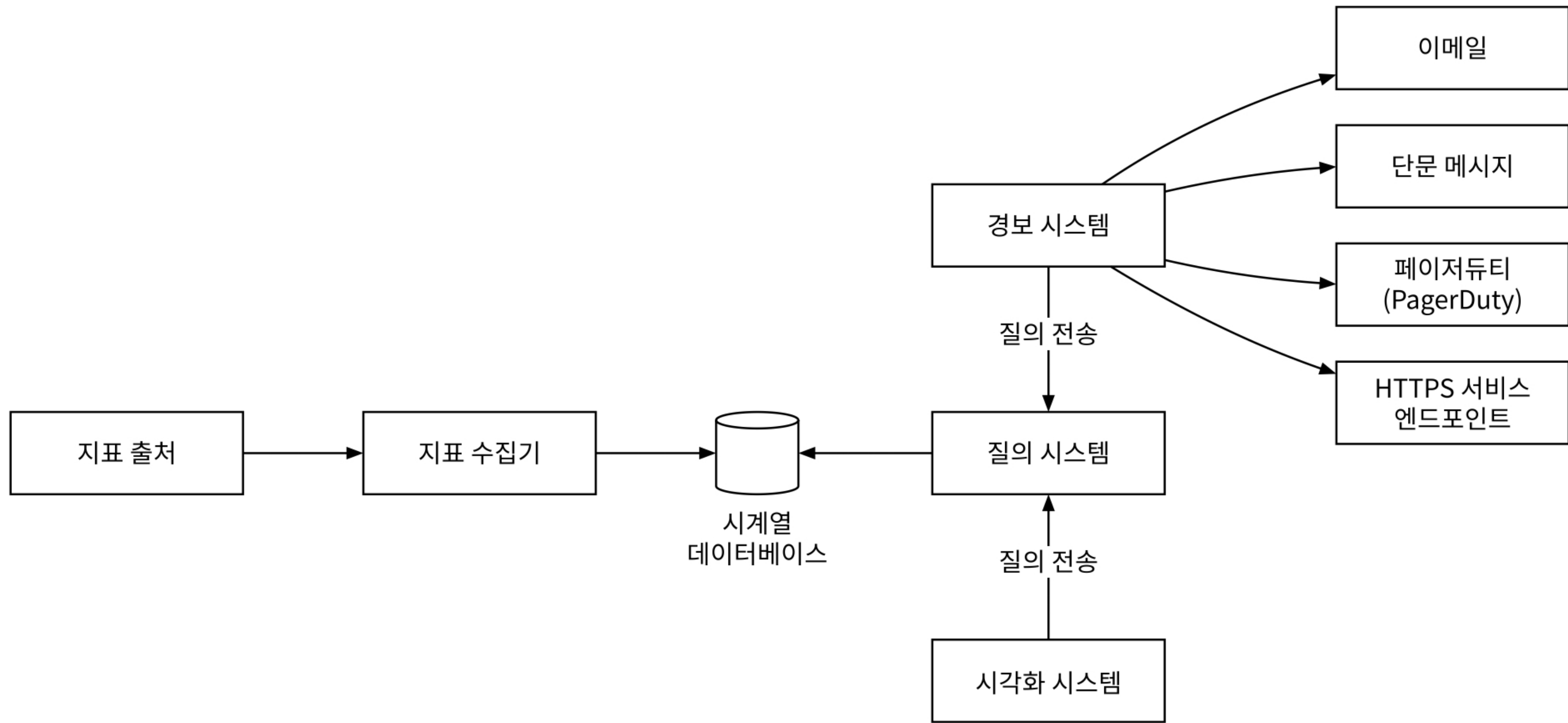


그림 5.6 개략적 설계안

상세 설계

상세 설계

지표 수집 - 풀 VS 푸시 모델

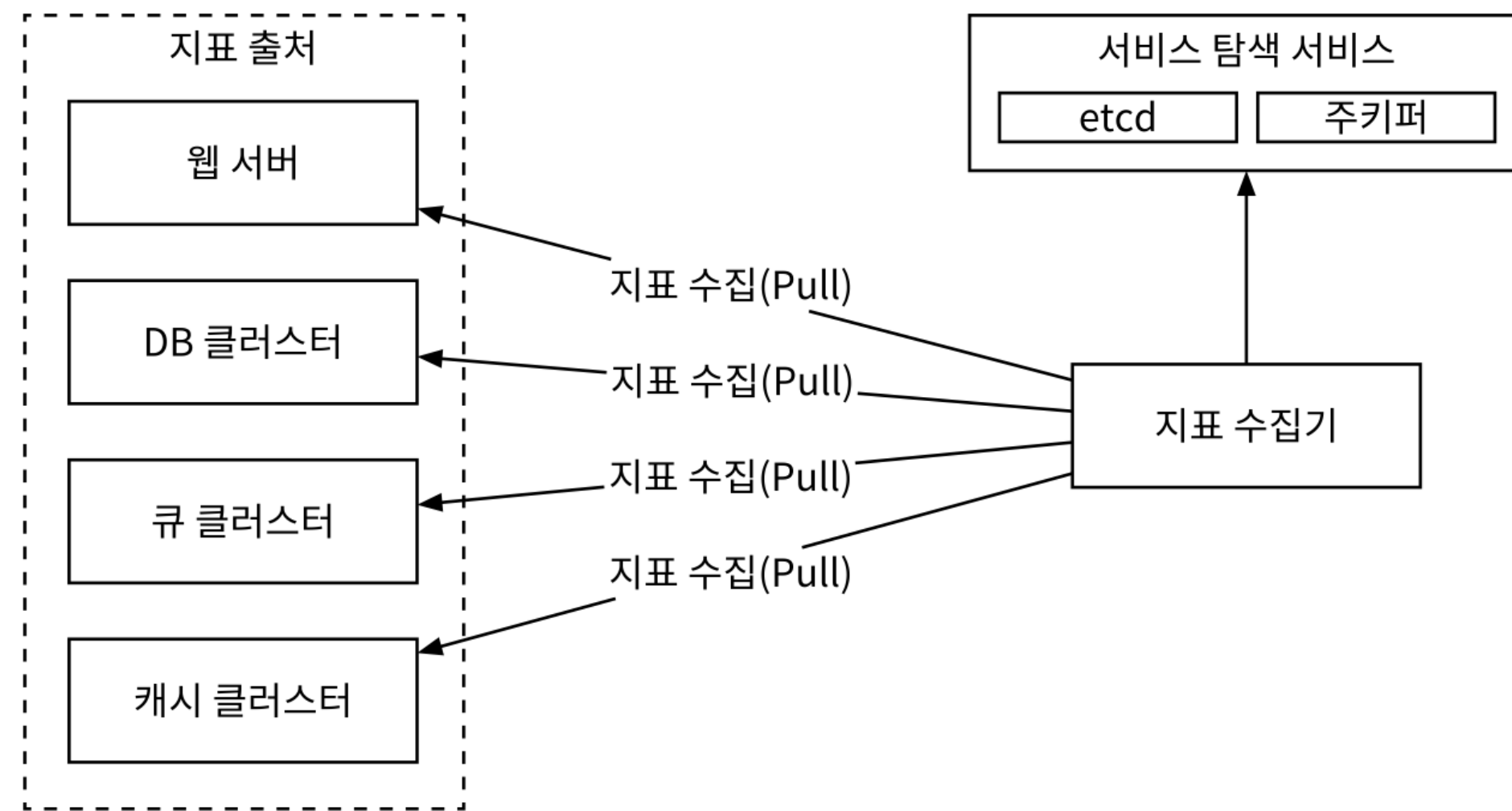


그림 5.8 풀 모델

VS

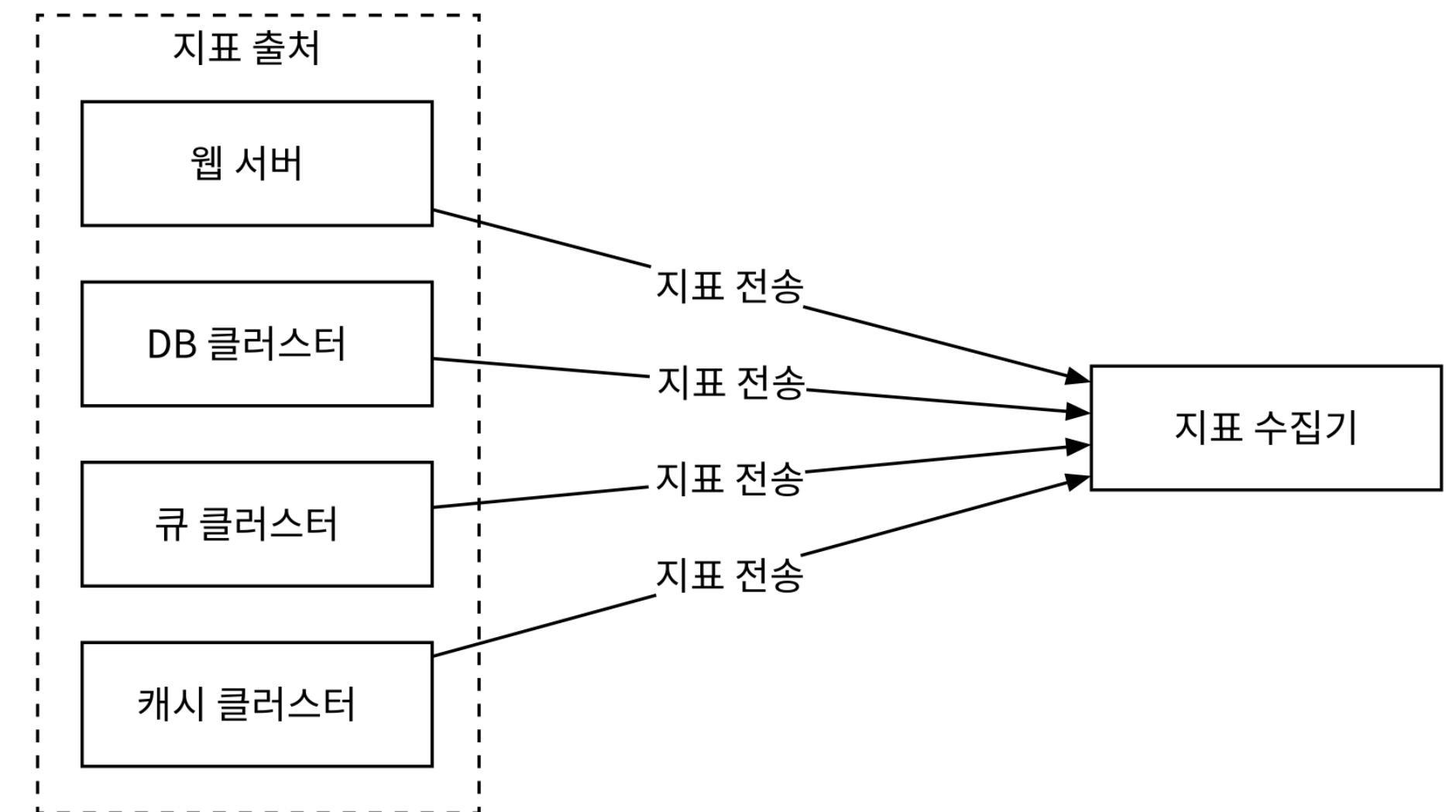


그림 5.12 푸시 모델

상세 설계

지표 수집 - 풀 모델

- 실행 중인 애플리케이션에서 주기적으로 지표 데이터를 가져오는 지표 수집기

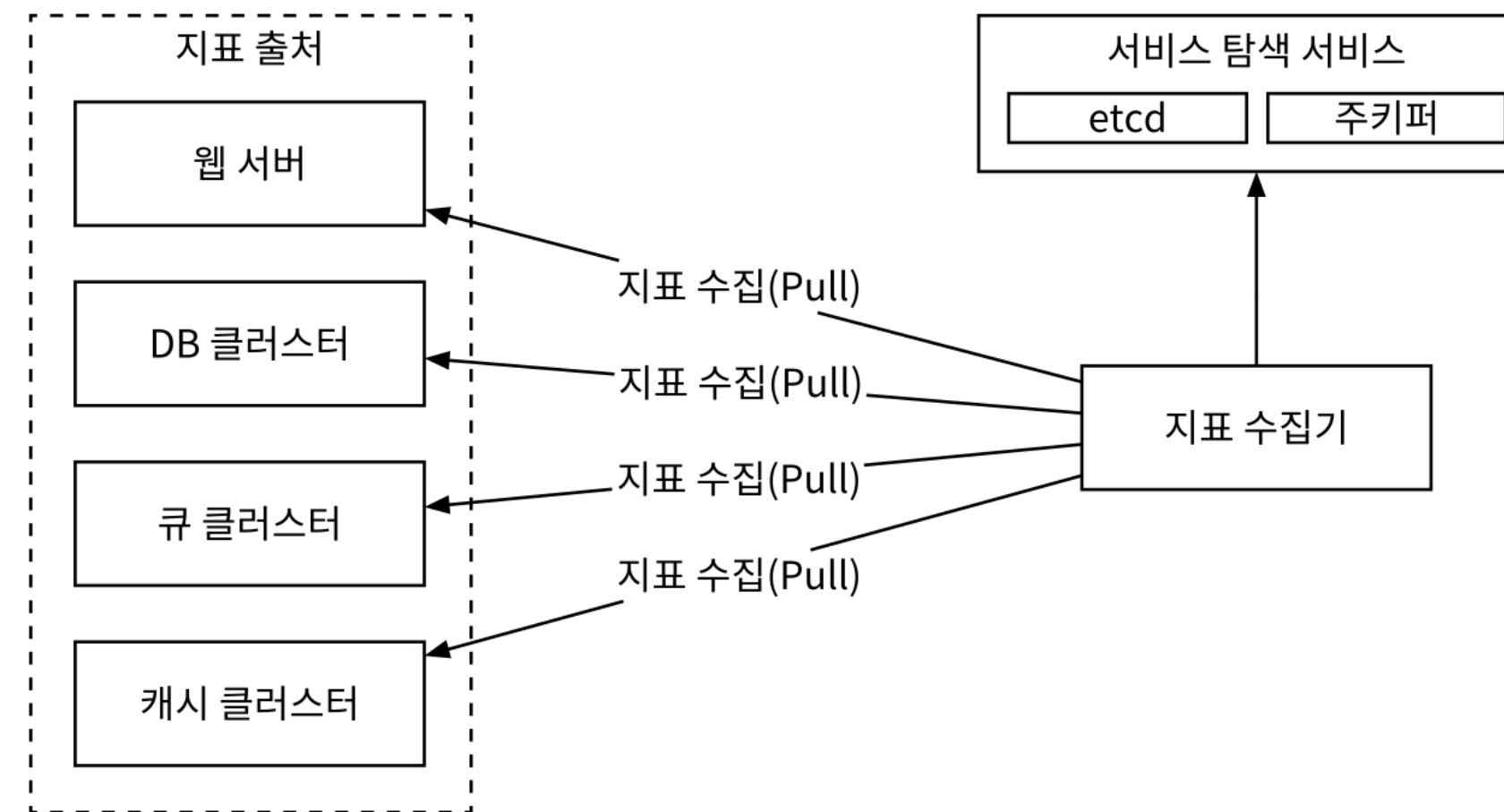


그림 5.8 풀 모델

상세 설계

지표 수집 - 풀 모델

1. 지표 수집기는 SDS에서 메타데이터 목록을 가져옴
2. 사전에 합의된 HTTP 엔드포인트에서 지표 데이터 가져옴
3. 엔드포인트 목록 변화 통지 받기 위한 변경 이벤트 알림 콜백을
서비스 탐색 컴포넌트에 등록 or 주기적으로 엔드포인트 목록 다시 가져오기

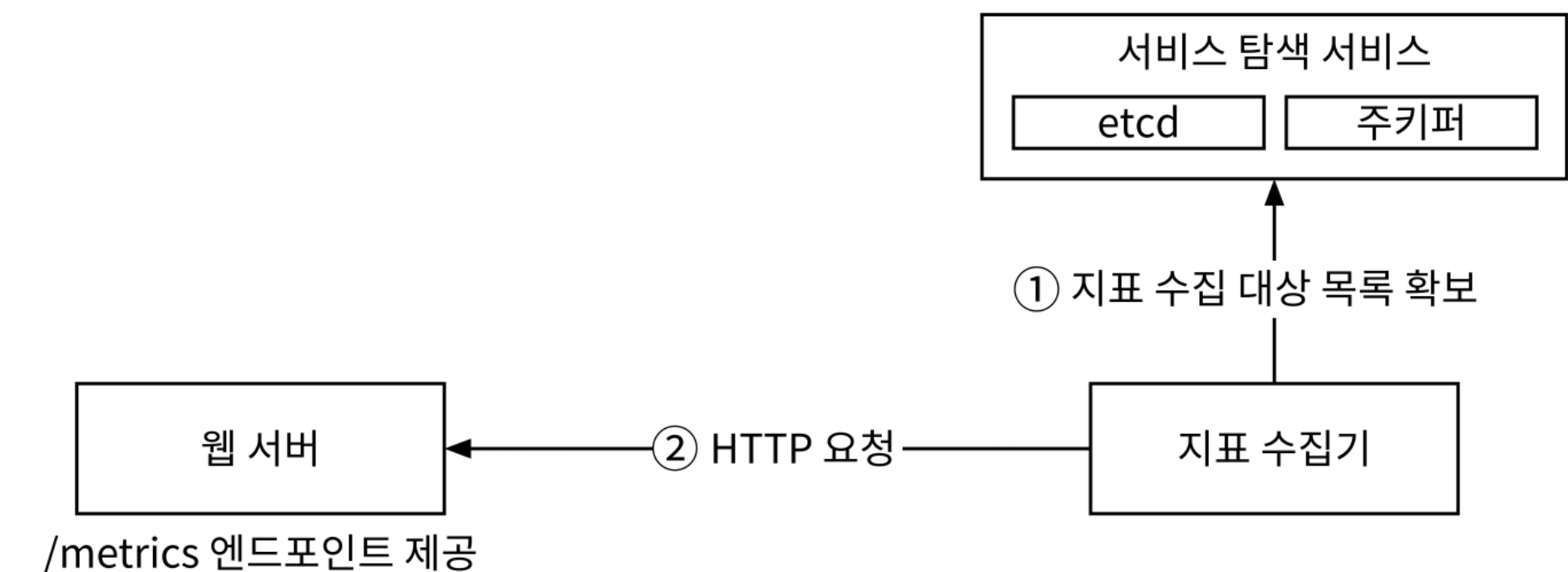


그림 5.10 서비스 탐색 기술 기반 풀 모델

상세 설계

지표 수집 - 풀 모델

- 지표 수집기 서버 한 대로는 부족 → 지표 수집기 서버 풀을 만들어야 데이터 규모 감당 가능
- 서버 여러 대 둘 때 문제: 같은 출처에서 데이터 중복해서 가져올 가능성 있음
→ 서버 간에 중재 매커니즘 필요
- 매커니즘: 안정 해시 링

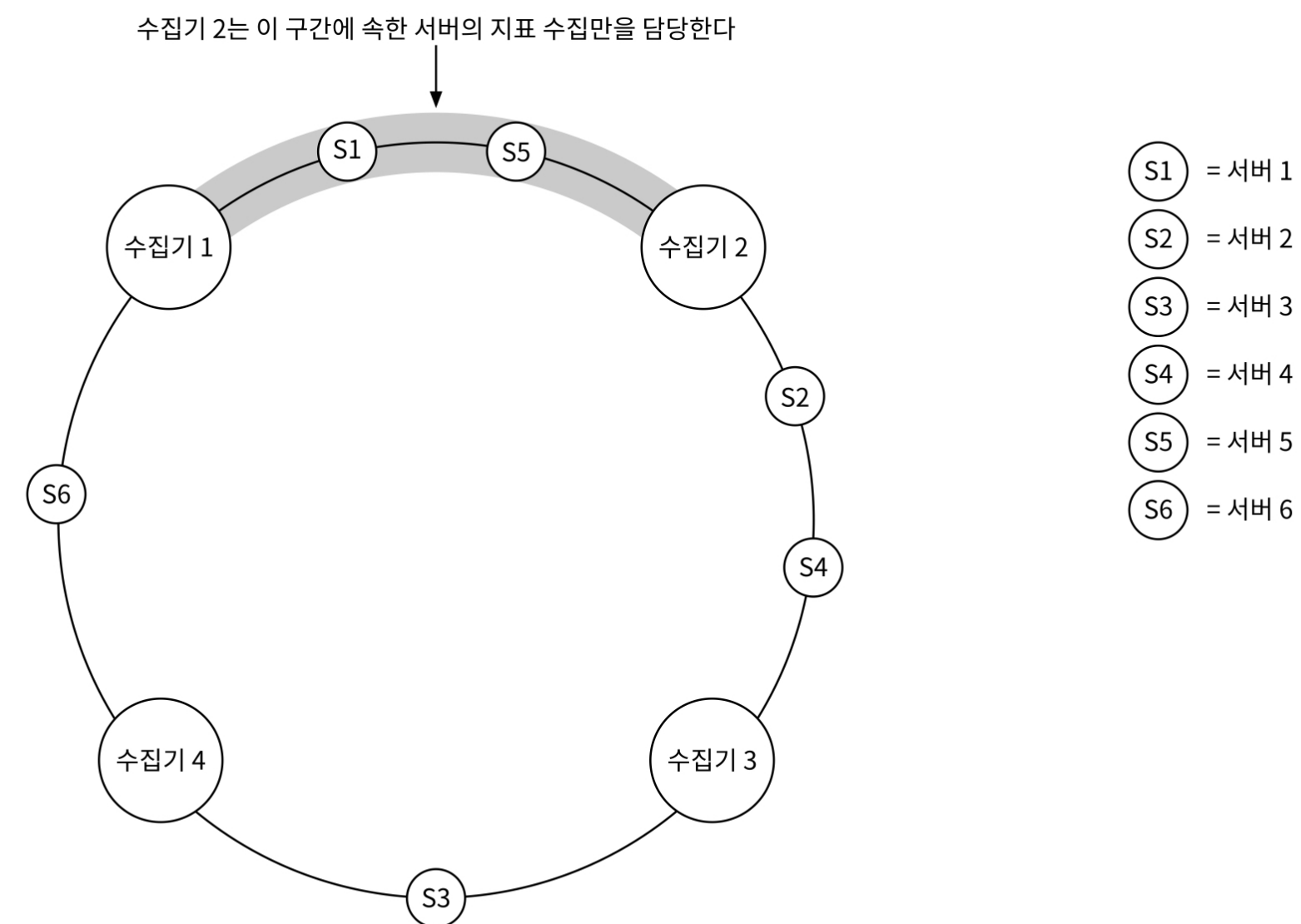


그림 5.11 안정 해시

상세 설계

지표 수집 - 푸시 모델

- 지표 출처 해당 서버가 직접 지표를 수집기에 전송
- 모니터링 대상 서버에 “수집 에이전트” 소프트웨어 설치

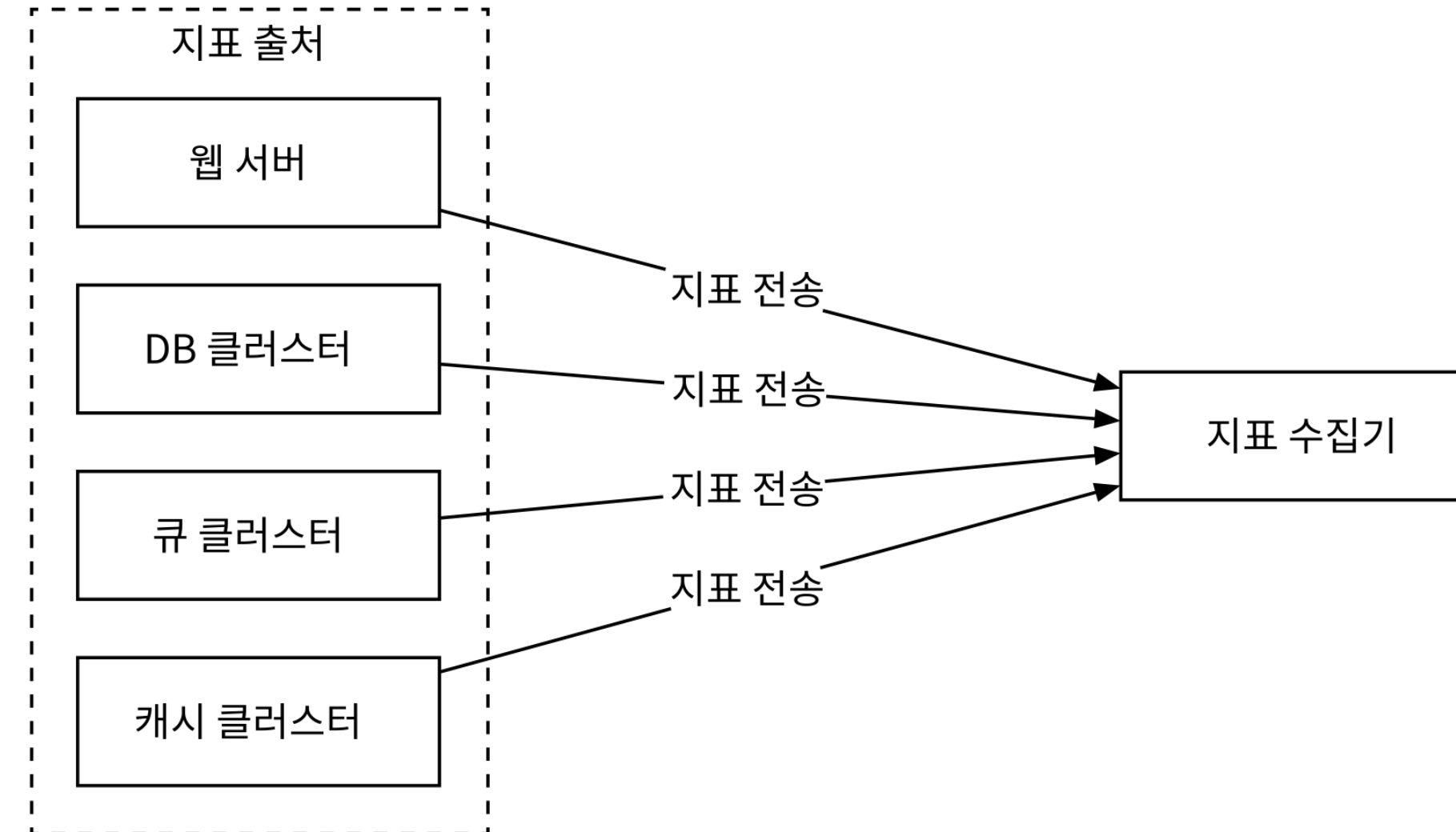


그림 5.12 푸시 모델

상세 설계

지표 수집 - 푸시 모델

- 에이전트가 위치한 서버 클러스터가 자동 규모 확장 가능 설정 → 데이터 소실 위험
- 지표 수집기 클러스터 자체 자동 규모 확장 가능 구성, 그 앞에 로드밸런서

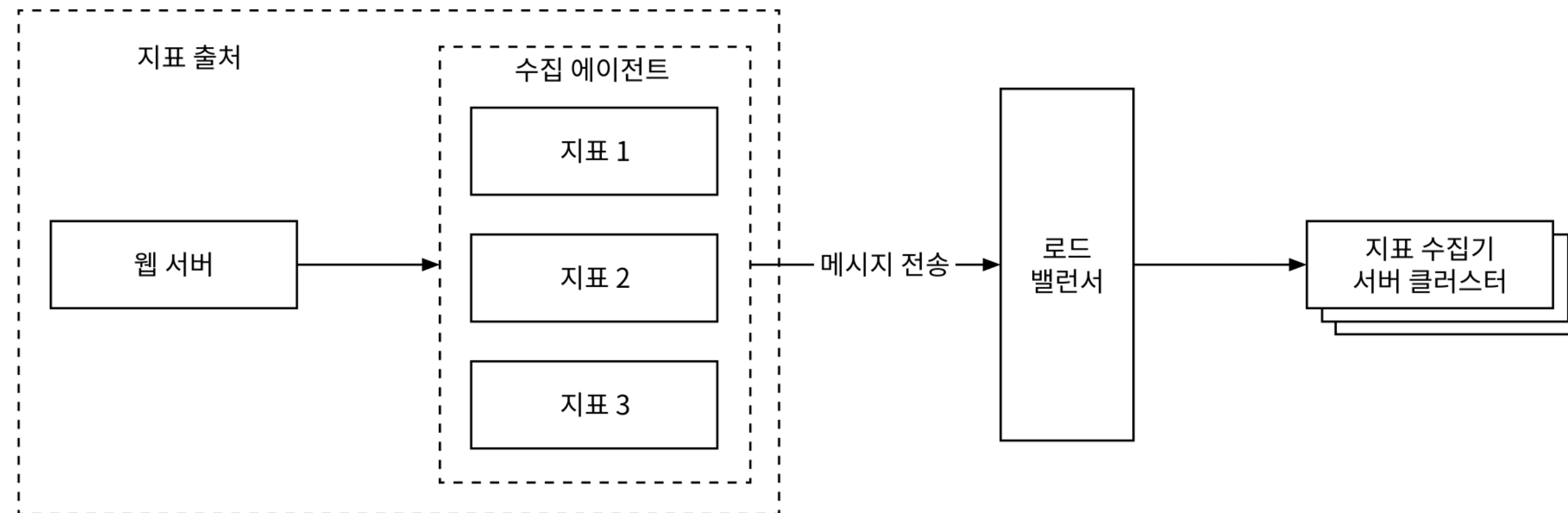


그림 5.13 로드밸런서 및 지표 수집기 클러스터의 자동 규모 확장

상세 설계

지표 수집 - 풀 VS 푸시 모델

- 정답 없다!
- 풀 모델: 프로메테우스
푸시 모델: 아마존 클라우드와치, 그래파이트
- 두 모델 모두 지원 → 지표 수집 에이전트를 설치할 서버 존재하지 않을 수도 있다는 점 감안

상세 설계

풀 VS 푸시 모델 - 장단점 비교

항목	풀 모델	푸시 모델
디버깅	/metrics 엔드포인트로 언제든지 확인 가능 → 풀이 낫다	-
상태 진단	응답 없으면 바로 장애로 판단 가능 → 풀이 쉽다	수신 안 되면 원인(네트워크/서버)이 불분명
짧은 생존 프로세스	끌어가기 전에 종료될 수 있음	종료 전에 스스로 보낼 수 있음 → 푸시가 낫다 (풀도 푸시 게이트웨이로 보완 가능)
방화벽/네트워크 구성	모든 서버가 접근 허용돼야 해 복잡	어디서 오든 수집 가능 → 푸시가 낫다
성능	TCP 사용	UDP 사용해 전송 지연 낮음 (TCP 연결 오버헤드 반론 있음)
데이터 신빙성	이미 정해진 서버 목록에서만 수집 → 신뢰 가능	아무나 보낼 수 있어 목록 관리·인증 필요

상세 설계

지표 전송 파이프라인의 규모 확장

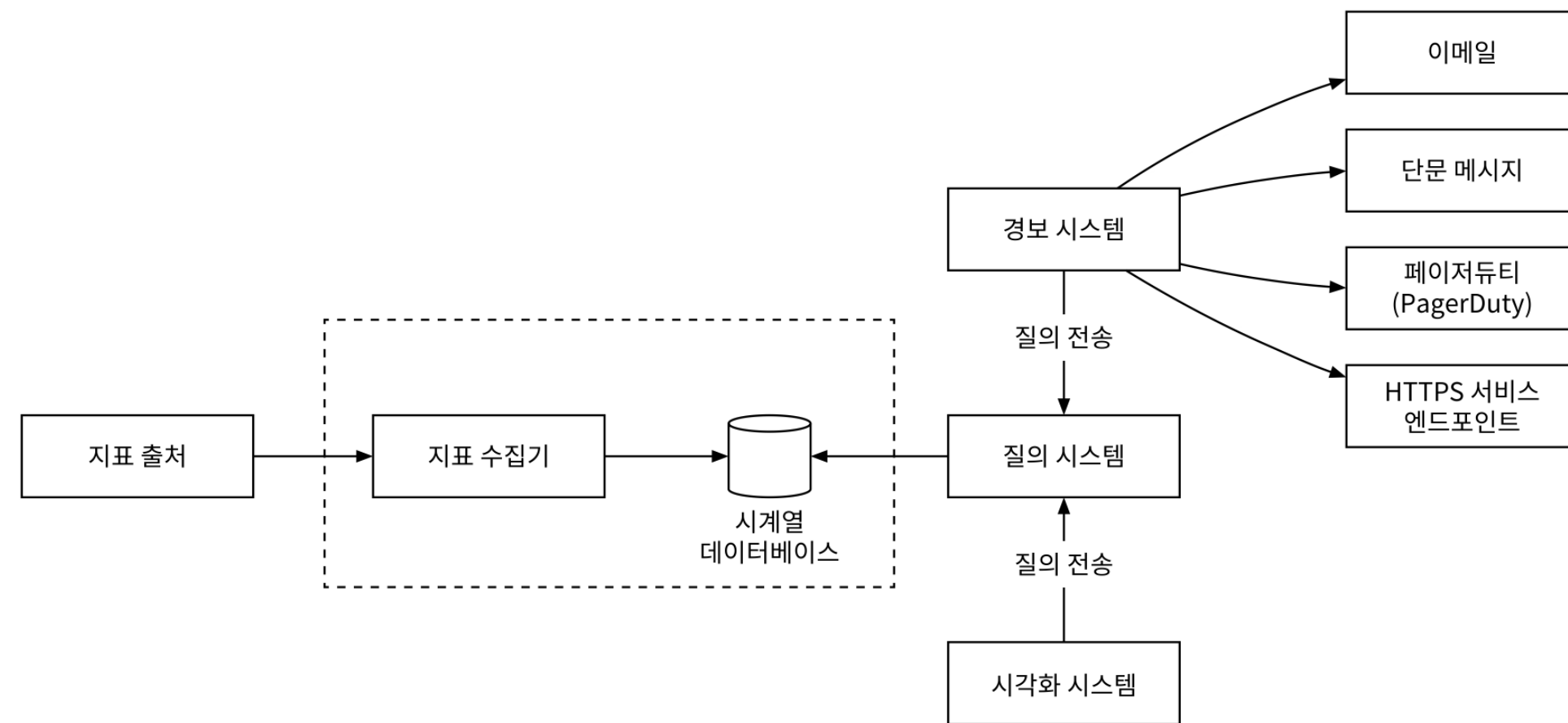


그림 5.14 지표 전송 파이프라인

- 시계열 데이터베이스에 장애 → 데이터 손실 위험

상세 설계

지표 전송 파이프라인의 규모 확장 - 카프카 사용

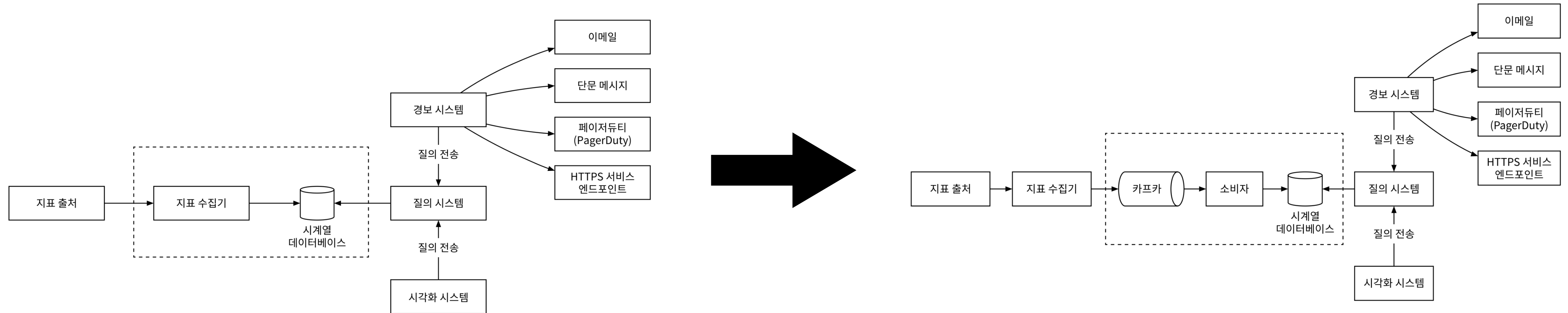


그림 5.14 지표 전송 파이프라인

그림 5.15 큐 추가

- 시계열 데이터베이스에 장애 → 데이터 손실 위험 → 큐 추가

상세 설계

지표 전송 파이프라인의 규모 확장 - 카프카 사용

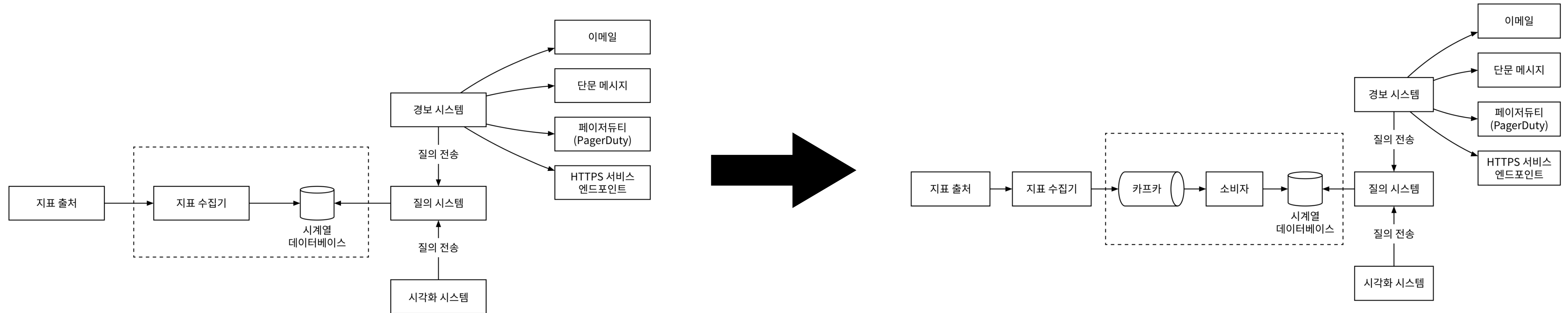


그림 5.14 지표 전송 파이프라인

그림 5.15 큐 추가

- 지표 수집기는 지표 데이터를 카프카와 같은 큐 시스템에 전송 → 소비자(스트림 처리 서비스, like 아파치 스톰, 플링크, 스파크)가 해당 데이터 받아 시계열 데이터베이스에 저장
- 장점
 - 카프카: 안정적, 규모 확장성
 - 데이터 수집 컴포넌트와 처리 컴포넌트 사이 결합도 ↓
 - 데이터베이스 장애 생겨도 데이터 소실 X (카프카에 보관)

상세 설계

지표 전송 파이프라인의 규모 확장 - 카프카의 대안

- 상용 규모의 카프카 시스템 구축 어려움
- 시장에는 큐 없이도 가능한 모니터링 시스템 존재 - 페이스북 고릴라

상세 설계

데이터 집계 지점

- 수집 에이전트가 집계 (클라이언트 측 집계)
- 데이터 수집 파이프라인에서 집계 (저장소 기록 전에 집계)
- 질의 시에 집계 (저장소 기록 후에 집계)

상세 설계

데이터 집계 지점

구분	수집 에이전트 집계 (클라이언트 측)	파이프라인 집계 (저장 전)	질의 시 집계 (저장 후)
집계 시점	클라이언트에서 보내기 전	저장소에 쓰기 전 (스트림 처리)	질의가 들어올 때
필요 기술	수집 에이전트 자체 로직	플링크 같은 스트림 프로세싱 엔진	저장소 + (선택) 캐시 계층
저장 데이터양	원본보다 약간 줄어듦 (분 단위 정도)	계산 결과만 저장 → 크게 감소	원본 그대로 전부 저장 → 가장 많음
집계 로직 복잡도	낮음 (복잡한 로직 지원 어려움)	중간~높음	자유로움 (원본 있으니 뭐든 재계산 가능)
늦게 도착한 데이터	-	처리 어려움	문제없음 (원본 다 있으니)
정밀도·유연성	낮음 (집계 후 원본 소실)	낮음 (원본 미보관)	높음 (원본 보존)
질의 응답 속도	빠름 (이미 집계됨)	빠름 (이미 집계됨)	느림 (그때 전체 데이터 계산)

- 속도와 정확성·유연성 사이의 “트레이드오프” 구조, 뭘 더 중요하게 보느냐에 따라 결정
- 에이전트 - 파이프라인 사이 차이: "얼마나 복잡한 집계 로직을 지원하는지"와 "저장 데이터양을 얼마나 줄이는지” (집계 로직 복잡도, 저장 데이터양)

상세 설계

질의 서비스 - 캐시 계층

- 질의 결과 저장할 캐시 서버 도입
- 질의 부하 ↓ 질의 서비스 성능 ↑

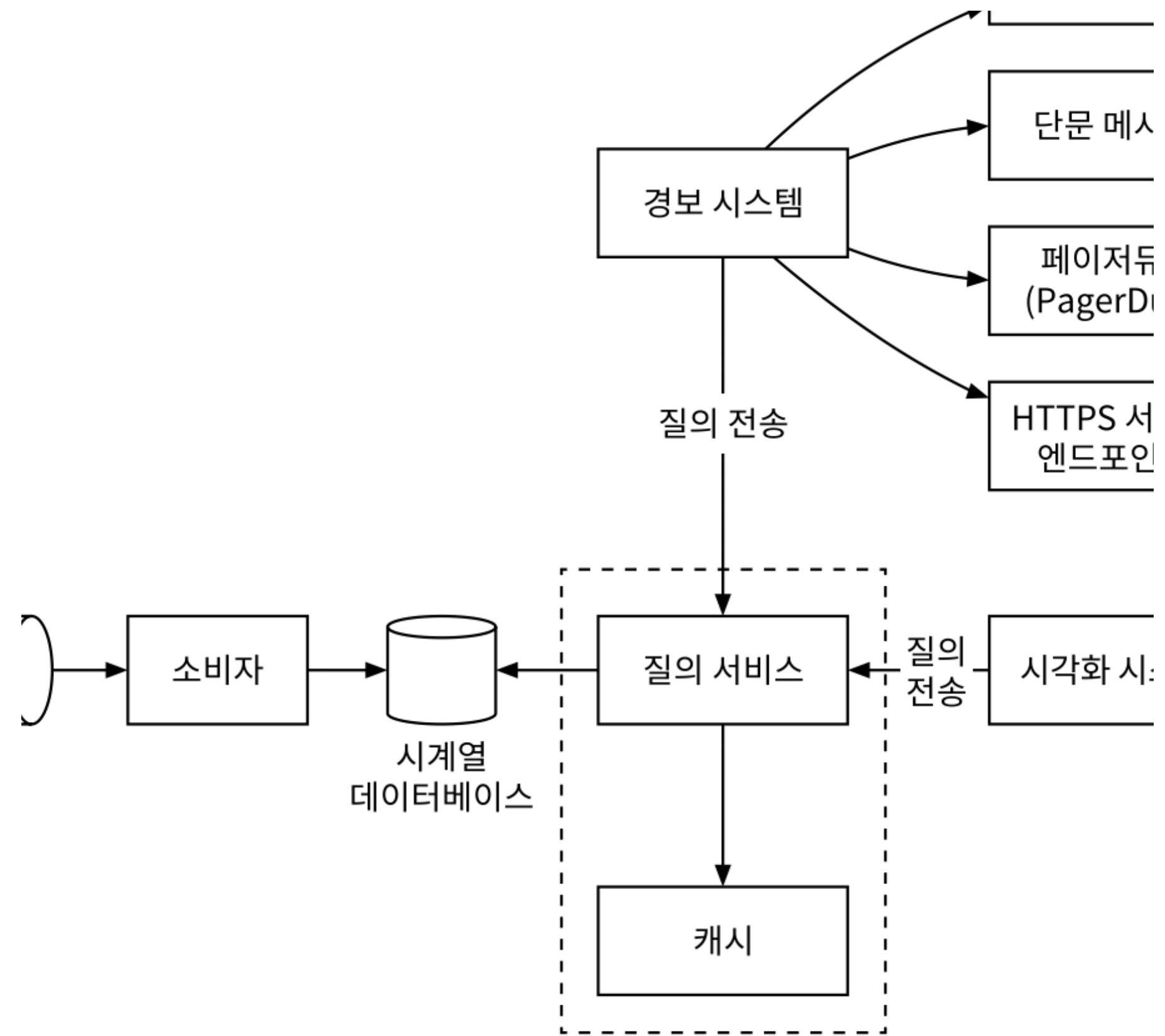


그림 5.17 캐시 계층

상세 설계

질의 서비스 - 두면 곤란한 경우

- 대부분의 상용 시각화 및 경보 시스템
이미 유명 시계열 데이터베이스와 연동 처리하는 강력한 플러그인 갖추고 있는 경우 많음
→ 별도 캐시 도입할 필요 없는 시계열 데이터베이스도 있음

상세 설계

저장소 계층 - 저장 용량 최적화

- 데이터 인코딩 및 압축: 크기 상당히 줄일 수 있음
- 다운 샘플링: 데이터 해상도 낮춰 저장소 요구량 줄임
- 냉동 저장소: 비활성 상태 데이터 보관

상세 설계

경보 시스템 & 시각화 시스템

- 대체적으로 직접 만들기보다 상용품 구입

상세 설계

경보 시스템 구성 방법

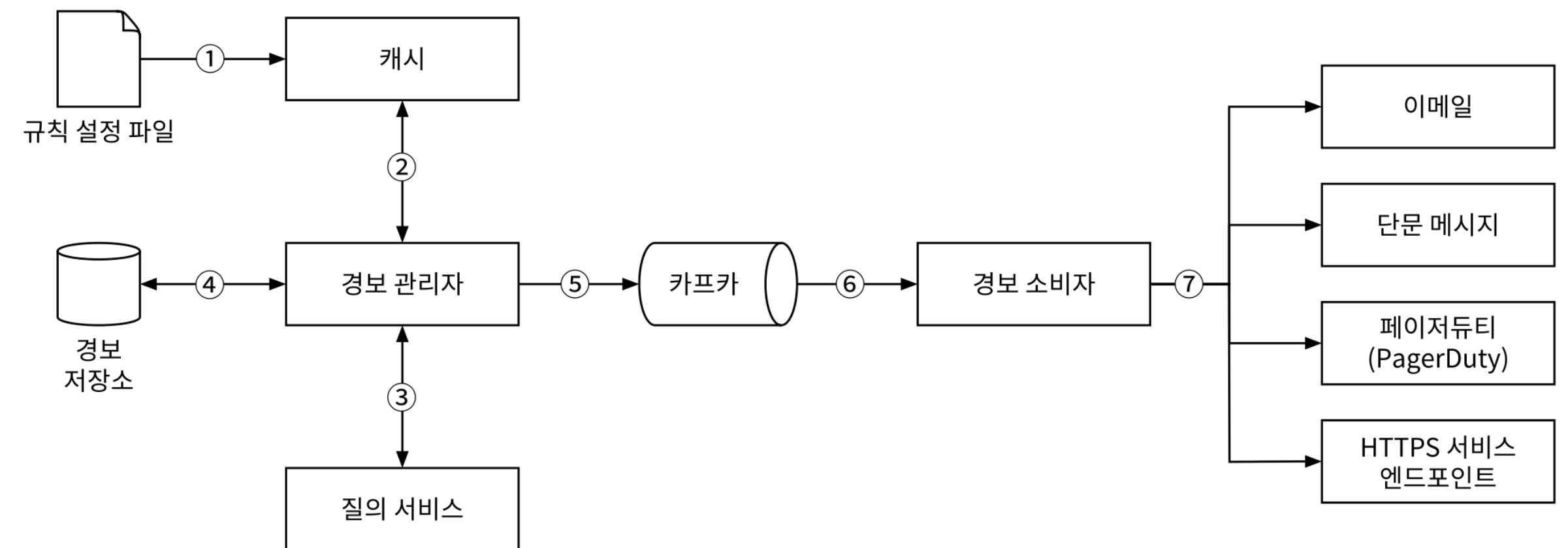


그림 5.19 경보 시스템

- 경보 처리 흐름

1. 설정 파일을 가져와 캐시 서버에 보관
2. 경보 관리자가 캐시에서 경보 설정 내역 가져옴
3. 경보 관리자가 규칙에 따라 주기적으로 질의 서비스를 호출
 - 3-1. 질의 결과가 임계값을 넘으면 경보 이벤트를 생성
4. 경보 저장소(카산드라 등)에 경보 상태를 저장해 최소 한 번 전달을 보장
5. 경보 이벤트를 카프카에 전달
6. 경보 소비자가 카프카에서 경보 이벤트를 읽음
7. 경보 소비자가 이메일, 단문 메시지, 페이저듀티, HTTP 엔드포인트 등으로 알림 전송

상세 설계

경보 시스템 - 만들 것인가 구매할 것인가

- 기업 필요로하는 규모 바로 지원 가능한 경보 시스템 시장에 많음.
대부분 유명 시계열 데이터베이스와 통합되어 있음 + 다양한 알림 채널 지원(이메일, 페이지듀티 등)
→ 경보 시스템 구현 X

상세 설계

시각화 시스템

- 데이터 계층 위에 만들어짐. 지표 대시보드
시각화 시스템 - 구현 어려움, 상용품 구입해서 사용



마무리

최종 설계안

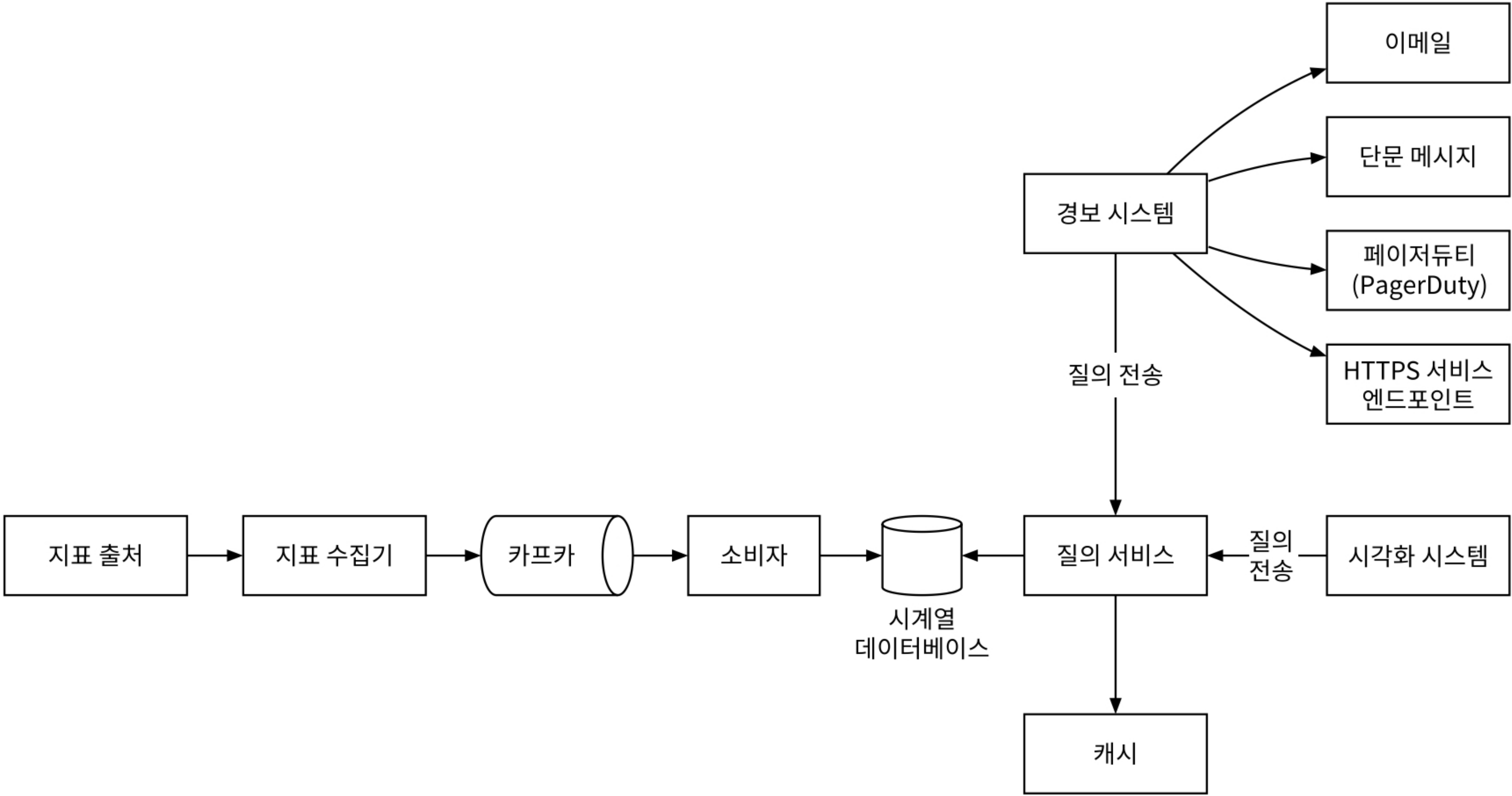


그림 5.22 최종 설계안

♡감사합니다♡

만든이: 한예진